

1. Implement the data link layer framing methods such as character stuffing and bit stuffing.

Character Stuffing:

```
def character_stuffing
(data): start_flag =
'@' end_flag = '#'
escape_char = '/'

stuffed_data = ""
for char in data:
if char in [start_flag, end_flag, escape_char]:
stuffed_data += escape_char # Escape special characters stuffed_data
+= char

framed_data = start_flag + stuffed_data + end_flag
return framed_data

# Example usage
data = "Hello/@World#"
stuffed = character_stuffing(data)
print("Character Stuffed Frame:", stuffed)
```

Bit Stuffing:

```
def bit_stuffing(data_bits):
stuffed = ""
count = 0
for bit in data_bits:
stuffed += bit
if bit == '1':
count += 1
if count == 5:
stuffed += '0' # Stuff a 0 after five 1s
count = 0
else:
count = 0
return stuffed
```

```
# Example usage
```

```
binary_data = "01111110111110"
```

```
stuffed = bit_stuffing(binary_data)
```

```
print("Bit Stuffed Data: ", stuffed)
```

OUTPUT :

Character Stuffed Frame:

@Hello//@World/##

Bit Stuffed Data: 0111110101111100

2. Write a C program to develop a DNS client server to resolve the given hostname.

```
// dns_client.c

#include <stdio.h>
#include <stdlib.h>
#include <netdb.h>
#include <string.h>
#include <arpa/inet.h>

int main() {
    char hostname[100];
    struct hostent *host_entry;
    char *IPbuffer;

    // Get hostname input
    printf("Enter hostname to resolve (e.g., www.google.com): ");
    scanf("%s", hostname);

    // Resolve hostname
    host_entry = gethostbyname(hostname);

    if (host_entry == NULL) {
        fprintf(stderr, "Could not resolve hostname.\n");
        return 1;
    }

    // Convert to IP string
    IPbuffer = inet_ntoa(*((struct in_addr*)host_entry->h_addr_list[0]));

    printf("Host name : %s\n", host_entry->h_name);
    printf("IP Address : %s\n", IPbuffer);

    return 0;
}
```

OUTPUT :

Enter hostname to resolve (e.g., www.google.com): www.google.com

Host name : www.google.com

IP Address : 142.250.184.100

3. Implement on a data set of characters the three CRC polynomials – CRC-12.

```
def crc12(data: bytes) -> int:
    """
    Compute CRC-12 (3GPP) for given byte data.
    Polynomial:  $x^{12} + x^{11} + x^3 + x^2 + x + 1$  (0x80F)
    """
    width = 12
    poly = 0x80F
    mask = (1 << width) - 1
    topbit = 1 << (width - 1)
    crc = 0x000
    for byte in data:
        crc ^= (byte << (width - 8))
        for _ in range(8):
            if crc & topbit:
                crc = ((crc << 1) ^ poly) & mask
            else:
                crc = (crc << 1) & mask
    return crc

# Example dataset
dataset = ["HELLO", "WORLD", "CRC", "DATA"]
for text in dataset:
    value = crc12(text.encode('utf-8'))
```

```
print(f"Input: {text!r}, CRC-12 = 0x{value:03X}")
```

OUTPUT :

Input: 'HELLO', CRC-12 = 0xD44

Input: 'WORLD', CRC-12 = 0xC7C

Input: 'CRC', CRC-12 = 0x8A6

Input: 'DATA', CRC-12 = 0xD82

4. Implement Dijkstra's algorithm to compute the shortest path in a graph.

```
import heapq
```

```
def dijkstra(graph, start):
```

```
    # Initialize distances with infinity
```

```
    distances = {vertex: float('inf') for vertex in graph}
```

```
    distances[start] = 0
```

```
    # Priority queue for the next node to visit
```

```
    pq = [(0, start)]
```

```
    while pq:
```

```
        current_distance, current_vertex = heapq.heappop(pq)
```

```
        # If we already found a shorter path before, skip
```

```
        if current_distance > distances[current_vertex]:
```

```
            continue
```

```
        # Explore neighbors
```

```
        for neighbor, weight in graph[current_vertex].items():
```

```
            distance = current_distance + weight
```

```
            # If a shorter path is found
```

```
if distance < distances[neighbor]:
    distances[neighbor] = distance
    heapq.heappush(pq, (distance, neighbor))
```

```
return distances
```

```
# Example graph (Adjacency List representation)
```

```
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'A': 4, 'C': 5, 'D': 10},
    'C': {'A': 2, 'B': 5, 'D': 3},
    'D': {'B': 10, 'C': 3}
}
```

```
# Run Dijkstra's Algorithm
```

```
start_node = 'A'
shortest_paths = dijkstra(graph, start_node)
```

```
# Output shortest distances
```

```
print(f"Shortest paths from {start_node}:")
for node, distance in shortest_paths.items():
    print(f" - {node}: {distance}")
```

OUTPUT :

Shortest paths from A:

- A: 0

- B: 4

- C: 2

- D: 5

5. Take an example subnet graph with weights indicating delay between nodes. Now obtain Routing table at each node using distance vector routing algorithm

```
#include<stdio.h>

void main()
{
    int i,j,d[20],dt[50][50],min=0,mini=0,p,q,y=0,nd[30];
    char r[30],n[30],l[30],x;
    printf("enter number of nodes\n");
    scanf("%d",&p);
    printf("enter nodes\n");
    for(i=1;i<=p;i++)
        scanf(" %c",&r[i]);
    printf("enter node for which you want to calculate routing table\n");
    scanf(" %c",&x);
    for(i=1;i<=p;i++)
    {
        if( x==r[i])
            y=i;
    }
    nd[y]=0;
    l[y]='-';
    printf("enter number of neighbours are there for %c node\n",x);
    scanf("%d",&q);
    printf("enter neighbours of %c node\n",x);
    for(i=1;i<=q;i++)
        scanf(" %c",&n[i]);
    printf("enter delays table\n");
    for(i=1;i<=p;i++)
    {
        for(j=1;j<=q;j++)
        {
            scanf("%d",&dt[i][j]);
        }
    }
    printf("enter delay from %c to each of its neighbour\n",x);
    for(i=1;i<=q;i++)
        scanf("%d",&d[i]);
    for(i=1;i<=p;i++)
```


5 8 4 6
routing vector table for A is

0	-
8	D
7	D
7	B
6	D

6.Take an example subnet of hosts. Obtain a broadcast tree for it.

```

#include<stdio.h> #define
MAX 10

int adjacencyMatrix[MAX][MAX]; int
numNodes;

// Function to display adjacent nodes of a given node void
displayAdjacentNodes(int rootNode) {
    int i;

    printf("\nAdjacent nodes of node %d:\n", rootNode);

    for (i = 1; i <= numNodes; i++) {
        if (adjacencyMatrix[rootNode][i] == 1 ||
adjacencyMatrix[i][rootNode] == 1) {
            printf("%d\t", i);
        }
    }

    printf("\n");
}

int main() {
    int i, j, rootNode;

    // Input number of nodes
    printf("Enter number of nodes: ");
    scanf("%d", &numNodes);

    // Input the adjacency matrix
    printf("Enter adjacency matrix:\n");
    for (i = 1; i <= numNodes; i++) {
        for (j = 1; j <= numNodes; j++) {
            printf("Is there a connection from node %d to node
%d (1=yes, 0=no): ", i, j);
            scanf("%d", &adjacencyMatrix[i][j]);
        }
    }

    // Input the root node
    printf("Enter root node (1 to %d): ", numNodes);
    scanf("%d", &rootNode);
}

```

```

    // Display adjacent nodes
    displayAdjacentNodes(rootNode);

    return 0;
}

```

OUTPUT:

```
$ gcc broadcast.c
```

```
$ ./a.out
```

```
Enter number of nodes: 5 Enter
```

```
adjacency matrix:
```

```

Is there a connection from node 1 to node 1 (1=yes, 0=no): 0
Is there a connection from node 1 to node 2 (1=yes, 0=no): 1
Is there a connection from node 1 to node 3 (1=yes, 0=no): 0
Is there a connection from node 1 to node 4 (1=yes, 0=no): 1
Is there a connection from node 1 to node 5 (1=yes, 0=no): 0
Is there a connection from node 2 to node 1 (1=yes, 0=no): 1
Is there a connection from node 2 to node 2 (1=yes, 0=no): 0
Is there a connection from node 2 to node 3 (1=yes, 0=no): 1
Is there a connection from node 2 to node 4 (1=yes, 0=no): 0
Is there a connection from node 2 to node 5 (1=yes, 0=no): 1
Is there a connection from node 3 to node 1 (1=yes, 0=no): 0
Is there a connection from node 3 to node 2 (1=yes, 0=no): 1
Is there a connection from node 3 to node 3 (1=yes, 0=no): 0
Is there a connection from node 3 to node 4 (1=yes, 0=no): 1
Is there a connection from node 3 to node 5 (1=yes, 0=no): 0
Is there a connection from node 4 to node 1 (1=yes, 0=no): 1
Is there a connection from node 4 to node 2 (1=yes, 0=no): 0
Is there a connection from node 4 to node 3 (1=yes, 0=no): 1
Is there a connection from node 4 to node 4 (1=yes, 0=no): 0
Is there a connection from node 4 to node 5 (1=yes, 0=no): 1
Is there a connection from node 5 to node 1 (1=yes, 0=no): 0
Is there a connection from node 5 to node 2 (1=yes, 0=no): 1
Is there a connection from node 5 to node 3 (1=yes, 0=no): 0
Is there a connection from node 5 to node 4 (1=yes, 0=no): 1
Is there a connection from node 5 to node 5 (1=yes, 0=no): 0

```

```
Enter root node (1 to 5): 4
```

```
Adjacent nodes of node 4:
```

```
1 3 5
```

7. Write a client-server application for chat using UDP**Server side:**

```
import socket

# 1. Create UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. Bind to a port
server_socket.bind(("127.0.0.1", 12345))
print("UDP Chat Server is running...")

while True:
    # 3. Receive message from client
    data, addr = server_socket.recvfrom(1024)
    message = data.decode()

    if message.lower() == "exit":
        print("Client has left the chat.")
        break

    print(f"Client: {message}")

    # 4. Get reply from server user
    reply = input("You: ")
    server_socket.sendto(reply.encode(), addr)

    if reply.lower() == "exit":
        print("Server is shutting down.")
        break

server_socket.close()
```

Client side :

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_address = ("127.0.0.1", 12345)

print("Type 'exit' to quit the chat")

while True:
    # 2. Take user input and send to server
    message = input("You: ")
    client_socket.sendto(message.encode(), server_address)

    if message.lower() == "exit":
```

```
print("You left the chat.")
break

# 3. Receive server reply
data, _ = client_socket.recvfrom(1024)
reply = data.decode()

if reply.lower() == "exit":
    print("Server has closed the chat.")
    break

print(f"Server: {reply}")

client_socket.close()
```

OUTPUT :

server.py

UDP Chat Server is running...

Client: Hello Server

You: Hi Client! How are you?

Client: I'm good, how about you?

You: I'm doing great!

Client: exit

Client has left the chat.

Server is shutting down.

client.py

Type 'exit' to quit the chat

You: Hello Server

Server: Hi Client! How are you?

You: I'm good, how about you?

Server: I'm doing great!

You: exit

You left the chat.

8.Implement programs using raw sockets (like packet capturing and filtering)

```

import struct

# Create a raw socket to capture all IP packets
# AF_INET = IPv4, SOCK_RAW = Raw socket, IPPROTO_IP = capture all protocols
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)

# Bind to your local machine IP
HOST_IP = "127.0.0.1" # Change to your machine's IP address
s.bind((HOST_IP, 0))

# Include IP headers in captured data
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# Enable promiscuous mode (Windows only)
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Capturing packets... Press Ctrl+C to stop.")

try:
    while True:
        packet, addr = s.recvfrom(65565)

        # Extract the IP header (first 20 bytes)
        ip_header = packet[0:20]
        iph = struct.unpack('!BBHHHBBH4s4s', ip_header)

        version_ihl = iph[0]
        version = version_ihl >> 4
        ihl = version_ihl & 0xF
        iph_length = ihl * 4
        ttl = iph[5]
        protocol = iph[6]
        src_addr = socket.inet_ntoa(iph[8])
        dest_addr = socket.inet_ntoa(iph[9])

        print(f"\nPacket: {src_addr} -> {dest_addr} | Protocol: {protocol} | TTL: {ttl}")

        # Filter example: show only ICMP packets (protocol 1)
        if protocol == 1:
            print("⚡ ICMP Packet Captured!")

```

```
except KeyboardInterrupt:
    print("\nStopped packet capture.")

finally:
    # Disable promiscuous mode (Windows only)
    s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    s.close()
```

OUTPUT:

Capturing packets... Press Ctrl+C to stop.

```
Packet: 127.0.0.1 -> 127.0.0.1 | Protocol: 6 | TTL: 128
Packet: 127.0.0.1 -> 127.0.0.1 | Protocol: 1 | TTL: 128
✂ ICMP Packet Captured!
Packet: 127.0.0.1 -> 127.0.0.1 | Protocol: 17 | TTL: 128
Packet: 127.0.0.1 -> 127.0.0.1 | Protocol: 6 | TTL: 128
Packet: 127.0.0.1 -> 127.0.0.1 | Protocol: 1 | TTL: 128
✂ ICMP Packet Captured!
```

9. Write a C program to perform the sliding window protocol.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TOTAL_FRAMES 10    // Total number of frames to send
#define WINDOW_SIZE 4      // Sender's window size
#define TIMEOUT_PROB 0.2   // Probability of frame loss (simulate errors)

void send_frame(int frame) {
    printf("Sender: Sending frame %d\n", frame);
}

int receive_ack(int frame) {
    // Randomly decide whether the frame is acknowledged or lost
    double r = (double)rand() / RAND_MAX;
    if (r < TIMEOUT_PROB) {
        printf("Receiver: Frame %d lost or ACK lost!\n", frame);
        return 0; // ACK not received
    }
    printf("Receiver: ACK %d received\n", frame);
    return 1; // ACK received
}

int main() {
    srand(time(NULL));
    int sent_upto = 0; // Last frame sent
    int ack_expected = 0; // Oldest unacknowledged frame

    while (ack_expected < TOTAL_FRAMES) {
        // Send frames in window
        while (sent_upto < ack_expected + WINDOW_SIZE && sent_upto <
            TOTAL_FRAMES) {
            send_frame(sent_upto);
            sent_upto++;
        }

        // Simulate receiving ACK for oldest frame if
        if (receive_ack(ack_expected)) {
            ack_expected++;
        } else {
            // Timeout: resend all frames from ack_expected
            printf("Timeout! Resending from frame %d\n", ack_expected);
            sent_upto = ack_expected;
        }
    }
}

```



```
}  
}
```

```
printf("All frames sent and acknowledged successfully!\n"); return 0;  
}
```

OUTPUT:

```
Sender: Sending frame 0  
Sender: Sending frame 1  
Sender: Sending frame 2  
Sender: Sending frame 3  
Receiver: ACK 0 received  
Sender: Sending frame 4  
Receiver: Frame 1 lost or ACK lost!  
Timeout! Resending from frame 1  
Sender: Sending frame 1  
Sender: Sending frame 2  
Sender: Sending frame 3  
Sender: Sending frame 4  
Receiver: ACK 1 received  
Receiver: ACK 2 received  
Receiver: ACK 3 received  
Sender: Sending frame 5  
Sender: Sending frame 6  
Receiver: Frame 4 lost or ACK lost!  
Timeout! Resending from frame 4  
Sender: Sending frame 4  
Sender: Sending frame 5  
Sender: Sending frame 6  
Sender: Sending frame 7  
Receiver: ACK 4 received  
Receiver: ACK 5 received  
Receiver: ACK 6 received  
Receiver: ACK 7 received  
Sender: Sending frame 8  
Sender: Sending frame 9  
Receiver: ACK 8 received  
Receiver: ACK 9 received  
All frames sent and acknowledged successfully!
```

10. Get the MAC or Physical address of the system using Address Resolution Protocol.

```
#!/usr/bin/env
python3 """
Get MAC address of a given IP using ARP (requires
scapy) """
from scapy.all import ARP, Ether, srp

def get_mac(ip):
    # Build ARP request (who-has)
    arp_req = ARP(pdst=ip)
    # Ethernet frame (broadcast)
    broadcast =
    Ether(dst="ff:ff:ff:ff:ff:ff") #
    Combine frame + request
    packet = broadcast / arp_req

    # Send packet, wait for response
    result = srp(packet, timeout=2, verbose=False)[0]

    for sent, received in result:
        return received.hwsrc # MAC

address return None

if __name__ == "__main__":
    target_ip = "192.168.1.1" # Example: Router
    IP mac = get_mac(target_ip)
    if mac:
        print(f"MAC address of {target_ip} is
{mac}") else:
        print(f"No response for {target_ip}")
```

Run as:

```
sudo python3 arp_mac.py
```

Example output:

MAC address of 192.168.1.1 is 14:cc:20:ab:98:11

11. Simulate the Implementing Routing Protocols using border gateway protocol (BGP)

Link-State Routing Protocols, such as **OSPF (Open Shortest Path First)**, use a map of the network topology to determine the best paths for data transmission. Unlike Distance Vector protocols, which rely only on hop count, OSPF considers multiple factors like bandwidth and cost.

Step-by-Step Procedure

Step 1: Open Cisco Packet Tracer

1. Launch **Cisco Packet Tracer** on your computer.

Step 2: Create a Network Topology

1. Add Devices

- Drag **three routers** (Router0, Router1, Router2) onto the workspace.
- Add **two switches** and **two PCs** (PC0 and PC1).

2. Connect Devices Using Cables

- Use **copper straight-through cables** to connect:
 - PC0 to Switch0
 - PC1 to Switch1
 - Switch0 to Router0
 - Switch1 to Router2
- Use **copper cross-over cables** to connect routers:
 - Router0 to Router1
 - Router1 to Router2

Step 3: Assign IP Addresses

Each router and end device needs an IP address.

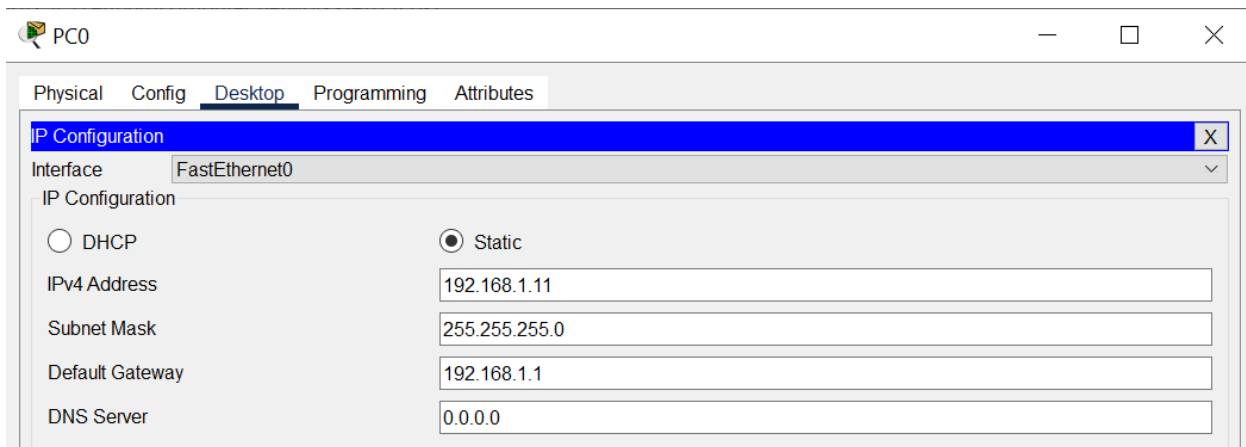
Configuring IP Addresses to PCs

- **PC0** → IP Address: **192.168.10.2**, Subnet Mask: **255.255.255.0**, Gateway: **192.168.10.1**
- **PC1** → IP Address: **155.168.1.2**, Subnet Mask: **255.255.255.0**, Gateway: **155.168.1.1**

OUTPUT:

PC IP configuration:-

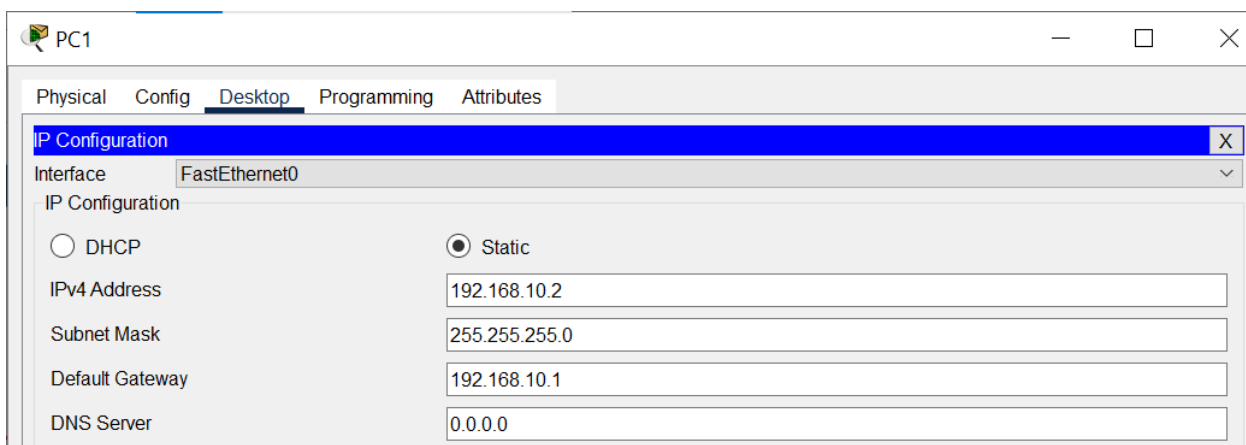
PC0/LAPTOP IPconfiguration



The screenshot shows the configuration window for PC0. The 'Desktop' tab is selected. Under 'IP Configuration', the 'Static' radio button is chosen. The fields are filled with: IPv4 Address: 192.168.1.11, Subnet Mask: 255.255.255.0, Default Gateway: 192.168.1.1, and DNS Server: 0.0.0.0.

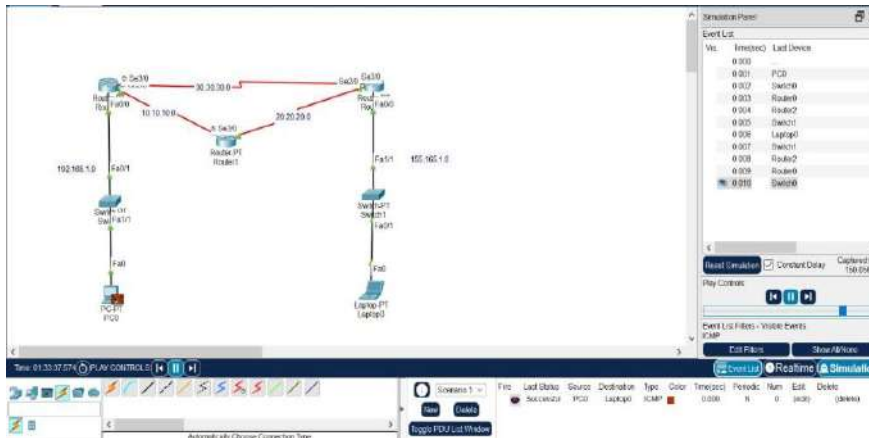
Interface	FastEthernet0
IP Configuration	
☐ DHCP	☒ Static
IPv4 Address	192.168.1.11
Subnet Mask	255.255.255.0
Default Gateway	192.168.1.1
DNS Server	0.0.0.0

PC1 IPconfiguration



The screenshot shows the configuration window for PC1. The 'Desktop' tab is selected. Under 'IP Configuration', the 'Static' radio button is chosen. The fields are filled with: IPv4 Address: 192.168.10.2, Subnet Mask: 255.255.255.0, Default Gateway: 192.168.10.1, and DNS Server: 0.0.0.0.

Interface	FastEthernet0
IP Configuration	
☐ DHCP	☒ Static
IPv4 Address	192.168.10.2
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	0.0.0.0



Router Configuration:-

At ROUTER 0:-

```
Router(config-if)#exit
Router(config)#router ospf 1
Router(config-router)#network 192.168.1.0 0.255.255.255 area 0
Router(config-router)#network 10.10.10.0 0.0.0.255 area 0
Router(config-router)#network 30.30.30.0 0.0.0.255 area 0
Router(config-router)#exit
```

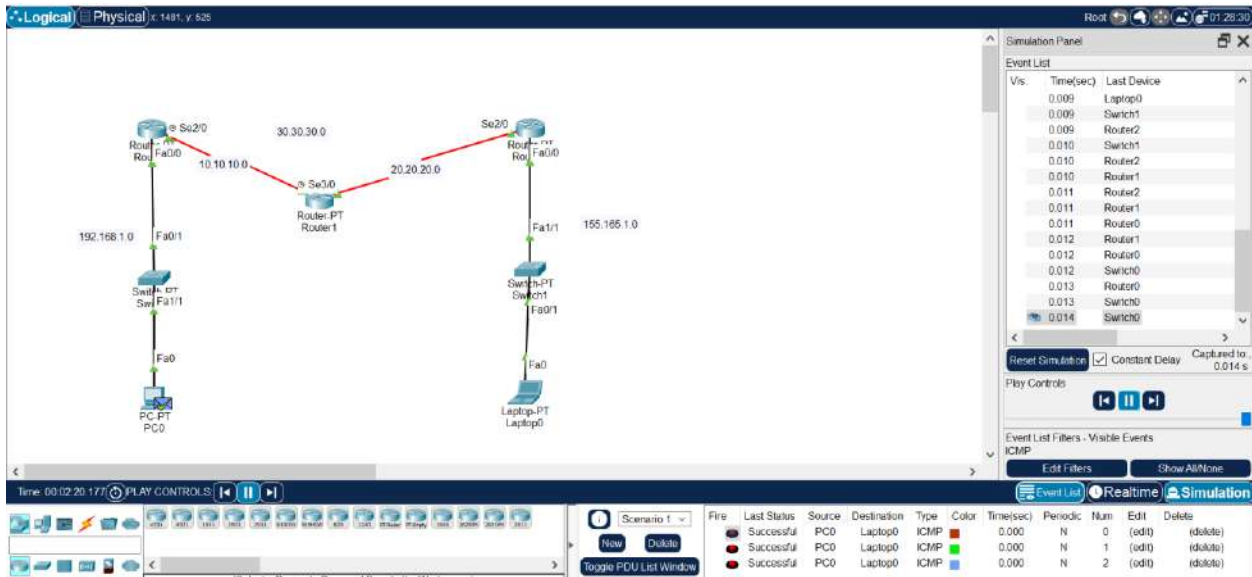
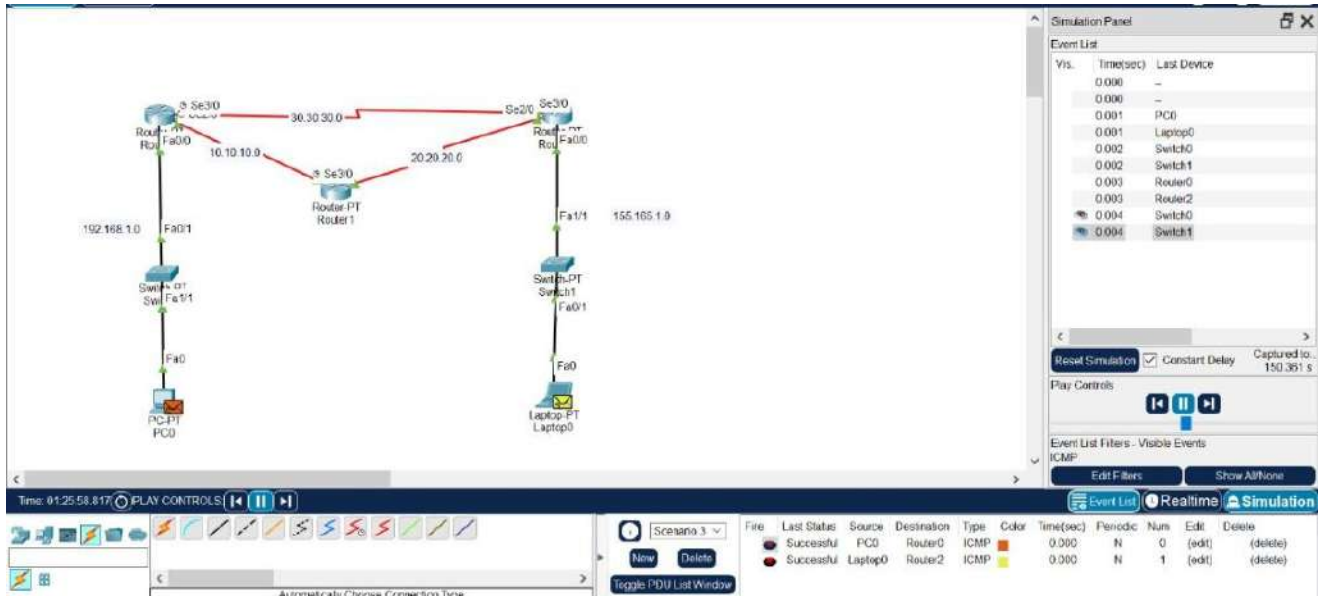
AT ROUTER 1:-

```
Router(config-if)#exit
Router(config)#router ospf 1
Router(config-router)#network 10.10.10.0 0.0.0.255 area 0
Router(config-router)#network 20.20.20.0 0.0.0.255 area 0
Router(config-router)#exit
```

AT ROUTER 2:-

```
Router(config-if)#exit
Router(config)#router ospf 1
Router(config-router)#network 30.30.30.0 0.0.0.255 area 0
Router(config-router)#network 155.165.1.0 0.0.255.255 area 0
Router(config-router)#exit
```

Final result:-



12. Simulate the **OPEN SHORTEST PATH FIRST** routing protocol based on the cost assigned to the path.

BGP: Border Gateway Protocol (BGP) is an Exterior Gateway Protocol (EGP) used on the internet to exchange routing information between autonomous systems (AS), which are networks under a single management entity.

Step-by-Step Procedure

Step 1: Open Cisco Packet Tracer

2. Launch **Cisco Packet Tracer** on your computer.

Step 2: Create a Network Topology

3. Add Devices

- Drag **2 routers** (Router0, Router1) onto the workspace.
- Add **two PCs** (PC0 and PC1).

4. Connect Devices Using Cables

- Use **copper straight-through cables** to connect:
 - PC0 to Router0
 - PC1 to Router1
- Use **copper cross-over cables** to connect routers:
 - Router0 to Router1

Step 3: Assign IP Addresses

Each router and end device needs an IP address.

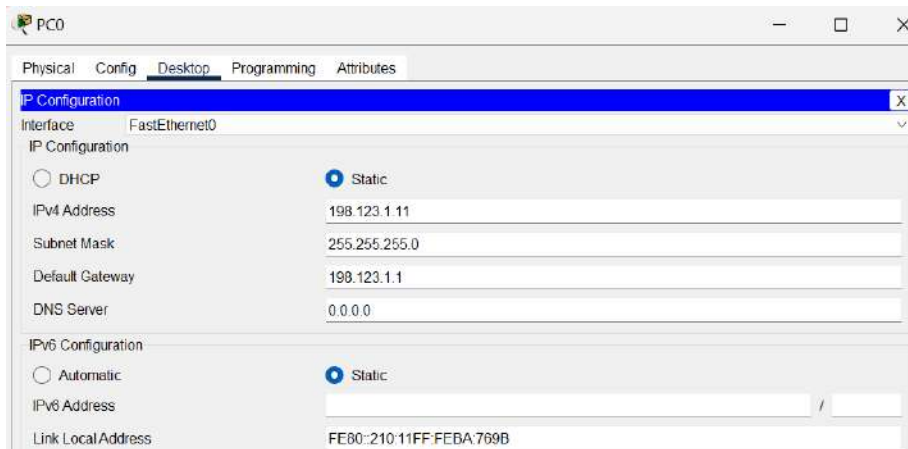
Configuring IP Addresses to PCs

- **PC0** → IP Address: **198.123.1.11**, Subnet Mask: **255.255.255.0**, Gateway: **198.123.1.1**
- **PC1** → IP Address: **155.123.3.11**, Subnet Mask: **255.255.255.0**, Gateway: **198.123.3.11**

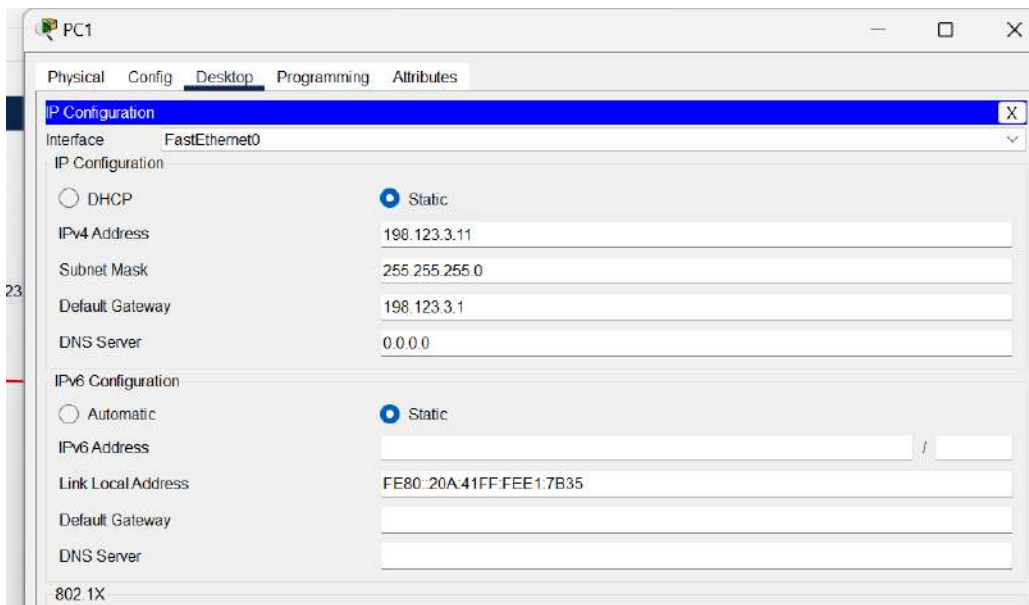
OUTPUT:

PC IP configuration:-

PC0 IPconfiguration

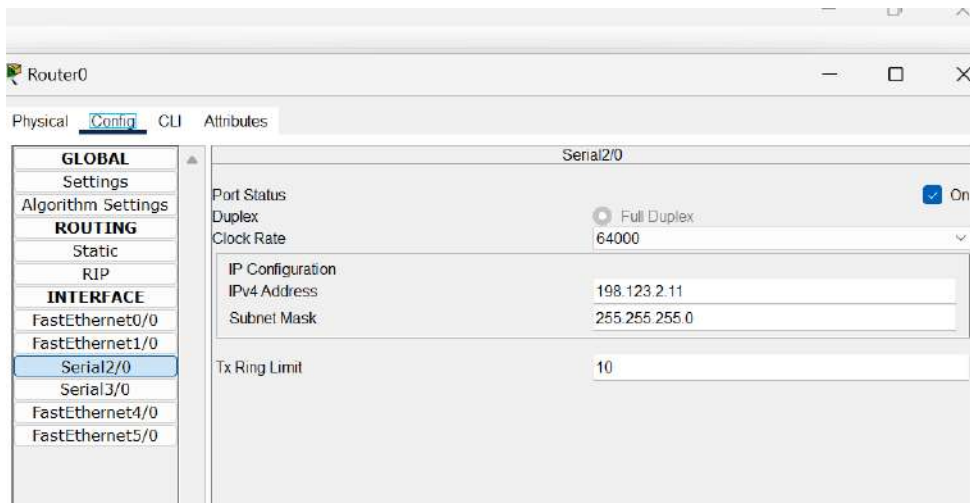
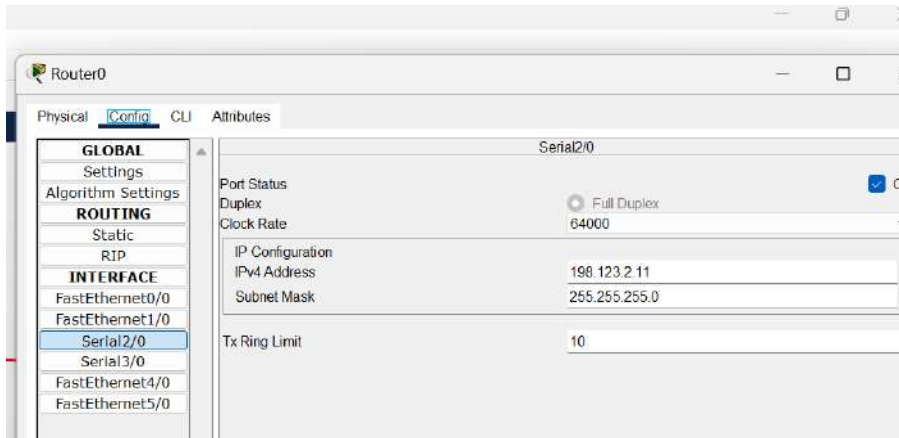


PC1 IPconfiguration



Router Configuration:-

At ROUTER 0:-



AT ROUTER 1:-

Router1

Physical **Config** CLI Attributes

GLOBAL

- Settings
- Algorithm Settings

ROUTING

- Static
- RIP

INTERFACE

- FastEthernet0/0**
- FastEthernet1/0
- Serial2/0
- Serial3/0
- FastEthernet4/0
- FastEthernet5/0

FastEthernet0/0

Port Status ☒ On

Bandwidth ☒ 100 Mbps ☐ 10 Mbps ☒ Auto

Duplex ☐ Half Duplex ☒ Full Duplex ☒ Auto

MAC Address 00D0.5806.D322

IP Configuration

IPv4 Address 198.123.3.1

Subnet Mask 255.255.255.0

Tx Ring Limit 10

Router1

Physical **Config** CLI Attributes

GLOBAL

- Settings
- Algorithm Settings

ROUTING

- Static
- RIP

INTERFACE

- FastEthernet0/0
- FastEthernet1/0
- Serial2/0**
- Serial3/0
- FastEthernet4/0
- FastEthernet5/0

Serial2/0

Port Status ☒ On

Duplex ☒ Full Duplex

Clock Rate 1200

IP Configuration

IPv4 Address 198.123.2.12

Subnet Mask 255.255.255.0

Tx Ring Limit 10

Commands at router 0:

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial2/0, ch
exit
Router(config)#router bgp 100
Router(config-router)#network 198.123.1.0
Router(config-router)#network 198.123.2.0
Router(config-router)#neighbor 198.123.2.12 remote-as 200
Router(config-router)#neighbor 198.123.3.11 remote-as-200
^
```

% Invalid input detected at '^' marker.

```
Router(config-router)#neighbor 198.123.3.11 remote-as 200
Router(config-router)#exit
```

Commands at router 1:

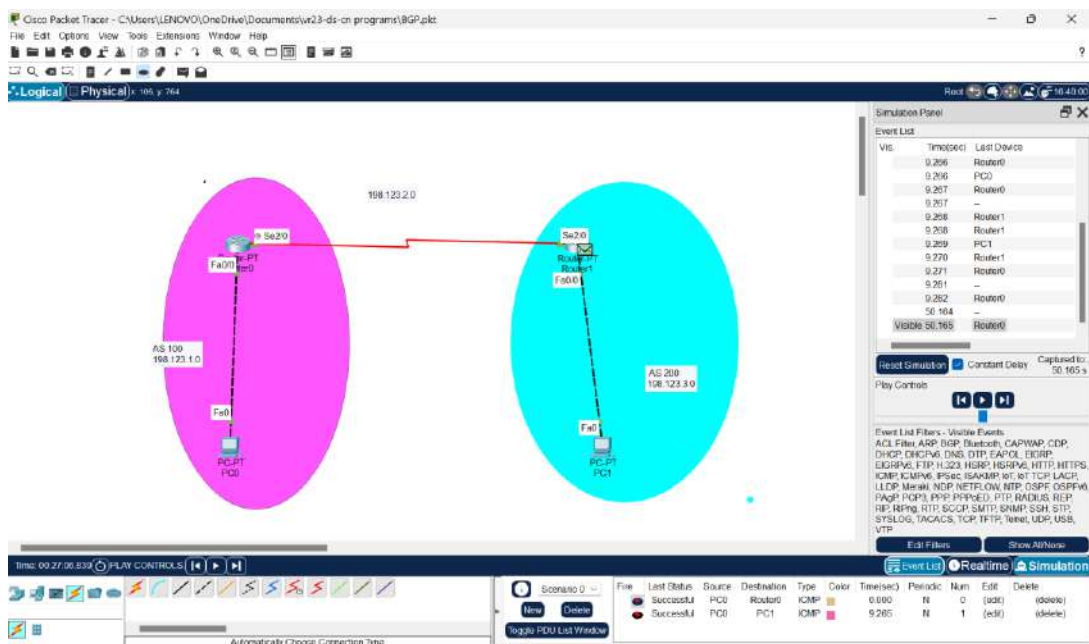
```

Router(config-if)#exit
Router(config)#router bgp 200
Router(config-router)#network 198.123.2.0
Router(config-router)#network 198.123.3.0
Router(config-router)#neighbor 198.123.2.11 remote-as 100
Router(config-router)%%BGP-5-ADJCHANGE: neighbor 198.123.2.11 Up
nennen
Router(config-router)#neighbhor 198.123.1.11 remote-as 100
^
% Invalid input detected at '^' marker.

Router(config-router)#neighbor 198.123.1.11 remote-as 100
Router(config-router)#exit

```

Final result



13. Install Wireshark Tool on PC and use into:

- a) Capture network traffic
- b) Determine default gateway address of your network
- c) Examine frame format and contents of Ethernet frames
- d) Filter and examine only ICMP traffic
- e) Run various network services like ping, ssh, dns ..etc and examine the traffic captured by Wire shark

Aim:

Install Wireshark Tool on PC and use into:

- a) Capture network traffic
- b) Determine default gateway address of your network
- c) Examine frame format and contents of Ethernet frames
- d) Filter and examine only ICMP traffic
- e) Run various network services like ping, ssh, dns etc. and examine the traffic captured by Wire shark

Description:

Wireshark is an open-source network protocol analyzer used to capture and examine data packets traveling over a network. It helps in monitoring network traffic, analyzing performance issues, and understanding communication between devices in real-time.

Step-by-Step Procedure:

Step 1: Install Wireshark Tool on PC

1. Download Wireshark from the official website.
2. Run the installation setup.
3. Install WinPcap or Npcap when prompted.
4. Launch Wireshark after installation.

Step 2: Capture Network Traffic

1. Open Wireshark and select the active network interface.
2. Click Start Capturing Packets.
3. Perform any network activity such as opening a website or pinging a server.
4. Stop the capture after observing the traffic.

Step 3: Determine Default Gateway Address

1. Open Command Prompt or Terminal.
2. Type the command to view network configuration.
3. Note the Default Gateway IP Address.
4. Use a filter in Wireshark to view traffic related to the gateway.

Step 4: Examine Ethernet Frame Format

1. Locate a captured Ethernet II frame.
2. Expand Frame, Ethernet II, and Internet Protocol sections.
3. Observe source, destination, and type fields.

Step 5: Filter and Examine ICMP Traffic

1. In Wireshark, enter the ICMP filter.
2. Run a ping command.
3. Observe the captured ICMP packets.

Step 6: Run Network Services and Examine Traffic

Perform various services and observe packets in Wireshark.

Service	Command/Action	Filter
Ping	ping command	icmp
SSH	Connect via SSH	tcp.port == 22
DNS	DNS lookup	dns

COMPUTER NETWORKS AND PROTOCOLS LAB MANUAL

Course Code: 1005233112

HTTP	Open website	http
HTTPS	Access secure site	tls

14. Simulate a three-node point-to-point network with a duplex link between them. Set the queue size and vary the bandwidth and find the number of packets dropped.

STEPS:

Step1: Select the hub icon on the toolbar and drag it onto the working window.

Step2: Select the host icon on the toolbar and drag it onto the working window. Repeat this for another host icon.

Step3: Select the link icon on the toolbar and drag it on the screen from host (node 1) to the hub and again from host (node 2) to the hub. Here the hub acts as node 3 in the point-to-point network. This leads to the creation of the 3-node point-to-point network topology. Save this topology as a .tpl file.

Step4: Double-click on host (node 1), a host dialog box will open up. Click on Node editor and you can see the different layers- interface, ARP, FIFO, MAC, TCPDUMP, Physical layers. Select MAC and then select full-duplex for switches and routers and half duplex for hubs, and in log Statistics, select Number of Drop Packets, Number of Collisions, Throughput of incoming packets and Throughput of outgoing packets. Select FIFO and set the queue size to 50 and press OK. Then click on Add. Another dialog box pops up. Click on the Command box and type the Command according to the following syntax:
`stg [-t duration(sec)] [-p port number] HostIPAddr` and click OK.

Step 5: Double-click on host (node 2), and follow the same step as above with only change in command according to the following syntax: `rtg [-t] [-w log] [-p port number]` and click OK.

Step 6: Double click on the link between node 1 and the hub to set the bandwidth to some initial value say, 10 Mbps. Repeat the same for the other node.

Step 7: Click on the E button (Edit Property) present on the toolbar in order to save the changes made to the topology. Now click on the R button (RunSimulation). By doing so a user can run/pause/continue/stop/abort/disconnect/reconnect/submit a simulation. No simulation settings can be changed in this mode.

Step 8: Now go to Menu->Simulation->Run. Executing this command will submit the current simulation job to one available simulation server managed by the dispatcher. When the simulation server is executing, the user will see the time knot at the bottom of the screen move. The time knot reflects the current virtual time (progress) of the simulation case.

Step 9: To start the playback, the user can left-click the start icon(|>) of the time bar located at the bottom. The animation player will then start playing the recorded packet animation.

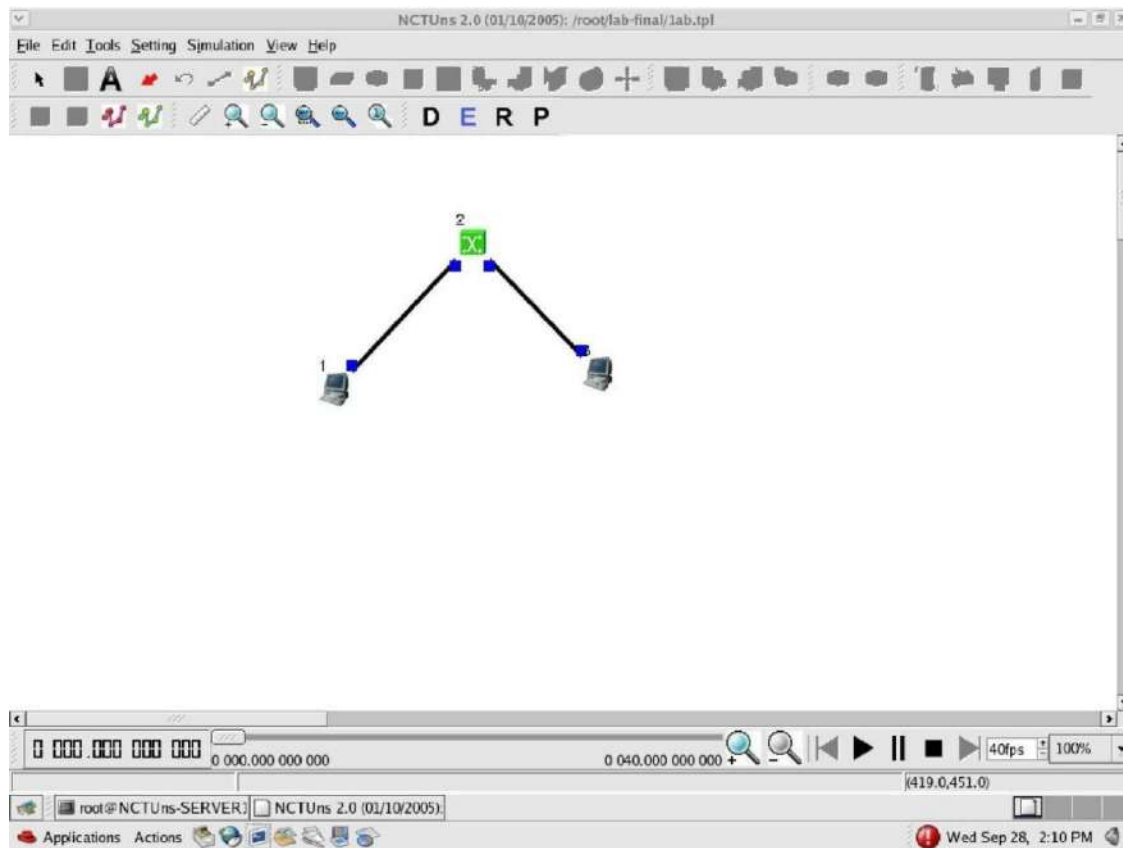
Step 10: Change the bandwidth say, 9 Mbps, and run the simulation and compare the two results.

Step 12: To view the results, go to the filename. results folder.

Note: To get the syntax of any command, double click on the host icon. Host dialog boxes appear and then choose App. Usage.

The screenshot below explain the topology.

OUTPUT:



15 Simulate a four-node point-to-point network and connect the link as follows: Apply a TCP agent between n0 to n3 and apply a UDP agent between n1 and n3. Apply relevant applications over TCP and UDP agents changing the parameters and determine the number of packets sent by two agents.

STEPS:

Step 1: Create the topology as specified in the question, in the draw mode of the simulator.

Step 2: Go to edit mode and save the topology.

Step 3: Setup a TCP connection between node 1 and node 3 using the following commands:

```
stcp [-p port] [-l writesize]  
hostIPaddr rtcp [-p port] [-l  
readsize]
```

Step 4: Setup a UDP connection between node 2 and node 3 using the following commands:

```
stg [-u payload size duration] [Host address]  
rtg [-u] [-w log]
```

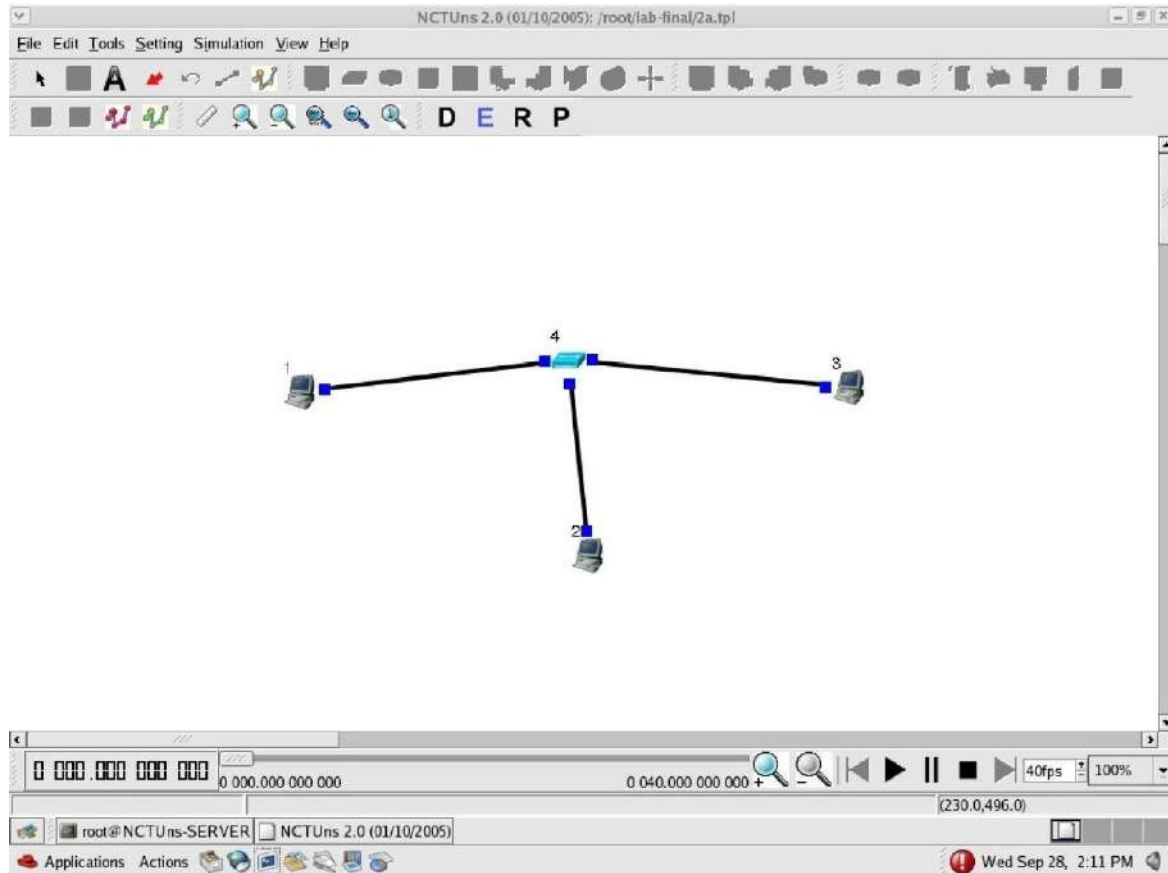
Step 5: Set the output throughput log to determine the number of packets sent by TCP/UDP as described in experiment 1.

Step 6: To view the results, go to the filename.results folder.

The screenshot of the topology is shown below:

COMPUTER NETWORKS AND PROTOCOLS LAB MANUAL
Course Code: 1005233112

OUTPUT:



16 .Simulate the transmission of ping messages over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

STEPS:

Step 1: Click on the subnet icon on the toolbar and then click on the screen of the working window.

Step 2: Select the required number of host sand a suitable radius between the host and the switch.

Step 3: In the edit mode, get the IP address of one of the hosts say, host 1 and then for the other host say, host2 set the drop packet and no: of collisions statistics as described in the earlier experiments.

Step 4: Now run the simulation.

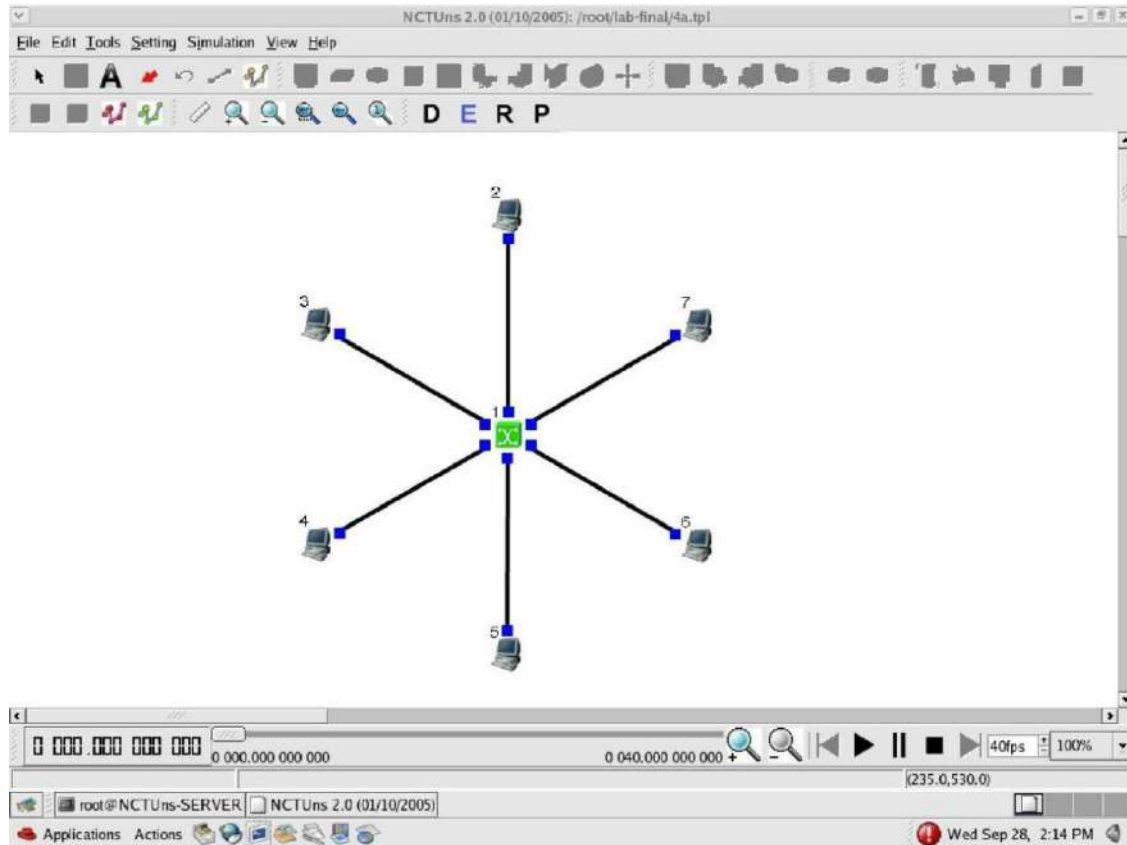
Step 5: Now click on any one of the hosts and click on command console and ping the destination node. ping IP Address of the host

Note: The no: of drop packets are obtained only when the traffic is more in the network. For checking the no of packets dropped press ctrl+C

The screenshot of the topology is shown below:

COMPUTER NETWORKS AND PROTOCOLS LAB MANUAL
Course Code: 1005233112

OUTPUT:



17. Simulate an Ethernet LAN using N nodes (6-10), change error rate and data rate and compare throughput.

STEPS:

Step 1: Connect one set of hosts with a hub and another set of hosts also through a hub and connect these two hubs through a switch. This forms an EthernetLAN.

Step 2: Setup a TCP connection between a host on one hub and host on another hub using the following command:

```
stcp [-p port] [-l writesize] hostIPAddr rtcp [-p port] [-l readsize]
```

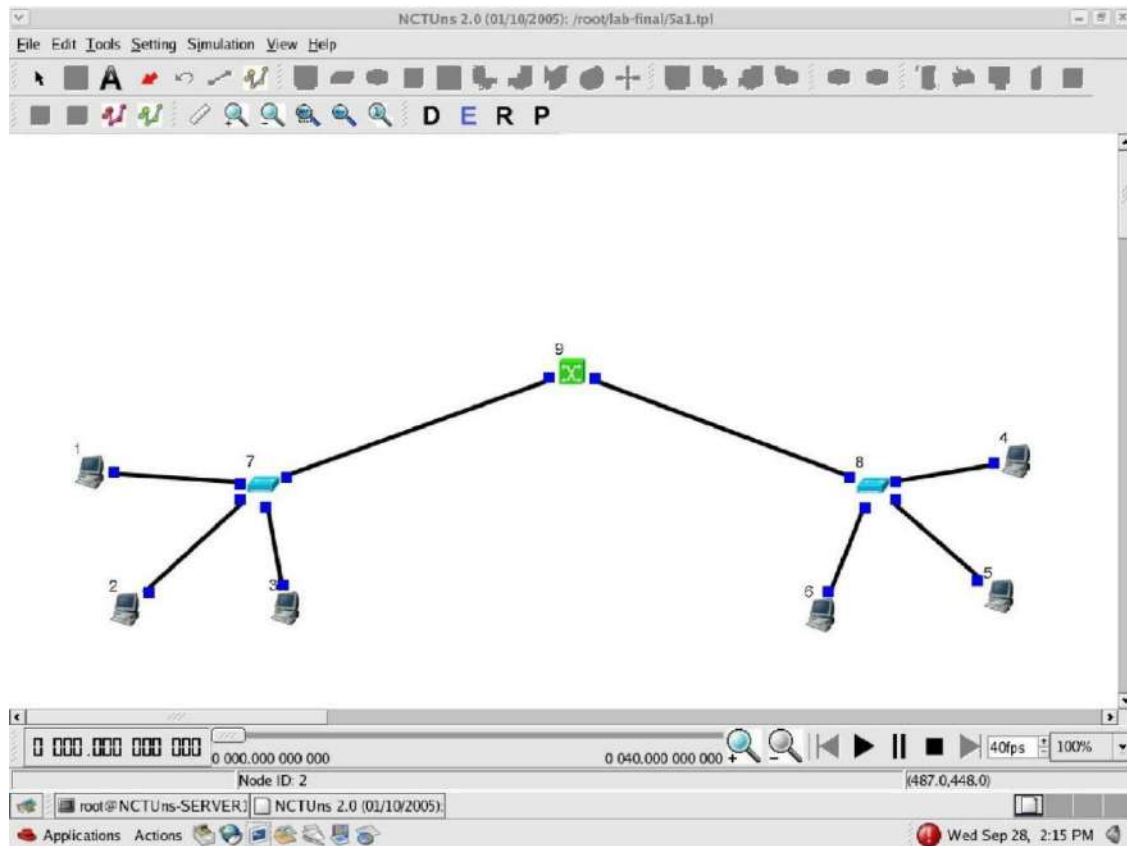
Step 3: Setup the error rate, data rate in the physical layer, input and output throughput in the mac layer as described above.

Step 4: Change error rate and data rate and compare the throughputs.

Step 5: View the results in the filename.results.

The screenshot of the topology is shown below:

OUTPUT:



18. Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and plot congestion window for different source/destination.* ns2/ns3/CISCO Packet Tracet/OPNET/any other network simulator may be used for simulation experiments.

STEPS:

Step 1: Connect one set of hosts with a hub and another set of hosts also through a hub and connect these two hubs through a switch. This forms an Ethernet LAN.

Step 2: Setup multiple traffic connections between the hosts on one hub and hosts on another hub using the following command:

`stcp [-p port] [-l writesize] hostIPaddr rtcp [-p port] [-l readsize]`

Step 3: Setup the collision log at the destination hosts in the MAC layer as described in the earlier experiments.

Step 4: To plot the congestion window go to Menu->Tools->Plot Graph->File->open->filename.results->filename.coll.log

Step 5: View the results in the filename.results.

The screenshot of the topology is shown below:

OUTPUT:

